

Mobile Testing Guidelines v3

Обновлено: 4 марта 2026

Зачем писать Unit-тесты

- Тесты пишем ради уверенности и скорости разработки, а не ради покрытия.
- Защита от регрессий.
- Безопасный рефакторинг.
- Проверка сложной бизнес-логики.
- Проверка async-поведения.

Что покрываем Unit-тестами

- Форматтеры и конверторы.
- Бизнес-правила.
- Состояние и ветвление.
- Обработку ошибок.
- Кэширование.
- Async-операции.

Что НЕ покрываем Unit-тестами

- UI-верстку.
- Простые DTO-модели.
- Сторонние библиотеки.
- Framework-код.
- Pass-through код без логики.

Хороший Unit-тест

- Проверяет поведение, а не реализацию.
- Не зависит от времени.
- Не зависит от сети.
- Не зависит от базы данных.
- Не зависит от порядка запуска.
- Читается как сценарий Given / When / Then.

Async рекомендации

- Использовать deterministic scheduler.

- Избегать `Task.sleep`.
- Избегать `DispatchQueue.asyncAfter`.
- Инжектировать `clock`.
- `MainActor` isolation.

Snapshot-тесты

- Использовать только для сложных UI-состояний.
- Не использовать при динамических датах.
- Не использовать при наличии анимаций.
- Не использовать при наличии random id.
- Обновлять снапшоты осознанно.

Архитектура и тестируемость

- Dependency Injection.
- Избегать singletons.
- Избегать static state.
- Инжектировать network client.
- Инжектировать storage.
- Инжектировать scheduler.

Test Pyramid

Тип теста	Что проверяет
Unit	Бизнес-логика
Integration	Взаимодействие слоев
Snapshot	UI-состояние
UI	Пользовательский сценарий

Когда тест стоит писать?

- Логика может сломаться.
- Есть ветвление.
- Есть состояние.
- Есть async.
- Есть бизнес-правило.
- Был баг ранее.

Рекомендация: проверяем, что метод вызвался (Protocol + Mock)

Чтобы тестировать факт вызова метода (и количество вызовов), удобно использовать протокол и класс-мок, который реализует этот протокол. Если «реальный» класс вам недоступен (например, из стороннего модуля), его можно «подписать» на тестовый протокол через extension. В протоколе фиксируем контракт тех методов, вызов которых хотим проверить.

- Кэширование: если данных нет в кэше — ожидаем, что будет вызван запрос на сервер и результат закэшируется.
- Кэш-hit: если данные уже есть в кэше — проверяем, что сетевой клиент не дергается лишний раз.
- Количество вызовов: можно проверять общее число вызовов метода (например, чтобы ловить лишние обновления экрана).
- Сценарий Given/When/Then: мок хранит счетчики/флаги; в Then утверждаем, что нужные методы вызывались ровно столько раз, сколько ожидаем.

Мини-шаблон (идея)

- Протокол: описываем методы (например, `fetch()`, `getFromCache()`, `saveToCache()`).
- Мок: реализует протокол и считает вызовы (например, `fetchCallCount`, `saveCallCount`).
- SUT: получает зависимости через DI.
- Тест: задаём состояние кэша, запускаем действие, проверяем счетчики и аргументы.

Регламент разработки

- Реализация логики.
- Unit-тесты.
- UI.
- Snapshot.
- Manual тестирование.